COMPREHENSIVE ENGINEERING REPORT

Distributed Recommender Simulator

Modernization, Recommendation Semantics, Reliability, and Million-Row Scalability

A before-and-after technical account of every implementation, correction, diagnosis, verification result, and remaining risk developed in the project workstream from 10 June through 13 June 2026.

Prepared for	Mohsen Alavi
Repository	github.com/aabhari/test
Working branch	v3.2
Current committed HEAD	cb47942 — Add Doc2Vec batch mode and experiment reset
Evidence date	13 June 2026, America/Toronto
Report status	Code-grounded audit with local build and stress-test verification

REPOSITORY STATE WARNING The three named commits are pushed on `origin/v3.2`. Most work performed after `cb47942`—including the major GUI redesign, recommendation target modes, million-row scaling, adaptive scheduling, heap controls, and optimized post-processing—exists in the current local working tree and is not yet represented by a clean commit.

Technical report • 13 June 2026

Abstract

This report documents a concentrated modernization of a Java/JADE distributed recommender-system simulator. The work began with correctness defects in multi-user recommendation execution and expanded into experiment isolation, user-interface redesign, recommendation-objective controls, optional graph rendering, lifecycle hardening, idle-resource correction, synthetic data generation, million-row ingestion, sparse model preparation, memory-aware parallelism, configurable Java heap sizing, and elimination of obsolete post-recommendation corpus rewrites. The governing constraint throughout was preservation of existing recommendation semantics when the user chooses Algorithm Default.

The final local system can generate and ingest an exactly 1,000,000-row six-field dataset, execute sparse SVM preparation without the original dense-ARFF failure mode, and run multiple model workers concurrently when heap capacity permits. A local stress test on the generated 1,000,000-row corpus observed 1,970 ms for a sequential two-model comparison versus 733 ms for two concurrent sparse SVM workers, a measured 2.69× speedup in that bounded test. Ingestion retained all 1,000,000 rows using approximately 347 MB of Java heap. These numbers validate the specific tests, not every end-to-end algorithm or dataset.

The work also exposed important semantic facts. Legacy scores are normalized so the top candidate is always displayed as 100; this is a relative ranking score, not necessarily a calibrated probability. SVM and MLP natively classify the field-0 followee account, while Similarity, K-Means, K-Means Euclidean, and Doc2Vec natively rank users. The new target-mode framework makes this distinction explicit and supplies controlled cross-type conversions, but those converted modes are hybrid recommendation objectives rather than the untouched native algorithms.

Executive Verdict

> **OVERALL RESULT** The local implementation is materially more correct, observable, scalable, and usable than the starting system. The default recommendation paths were deliberately preserved where possible. The largest remaining engineering risk is release hygiene: most post-commit changes are still mixed with generated datasets, caches, result logs, class files, and other runtime artifacts in a dirty working tree.

- Correctness: Doc2Vec batch now scores each selected target independently instead of copying the first target’s map.
- Experiment integrity: a dedicated reset action removes known transient outputs while preserving source data and archived results.
- Semantic clarity: users can select Algorithm Default, Similar Users, or Followee Accounts, with validation and typed result payloads.
- Operational control: graph visualization can be skipped without discarding accumulated node and edge state.
- Reliability: run identifiers isolate repeated simulations, idle agents block instead of polling, and Gephi projects are closed.
- Scalability: streaming parsing, indexed user lookup, sparse ARFF output, Doc2Vec memory mapping, and adaptive model admission address the million-row failure modes.
- Responsiveness: obsolete per-user full-corpus rescans and Dataset-Test rewrites are bypassed in favor of indexed post-processing.
- Release status: only commits `fe57855`, `6a00b58`, and `cb47942` are currently pushed to `origin/v3.2`.

Contents

Abstract and Executive Verdict 2

1. Scope, Method, and Evidence 4

2. Starting Architecture and Failure Surface 5

3. Git History and Delivery State 6

4. Correct Doc2Vec Batch Mode 7

5. Brand-New Experiment Reset 8

6. GUI Modernization and Interaction Repairs 9

7. Recommendation Target Modes 11

8. Optional Graph Output with Persistent Edge State 12

9. Repeated-run Lifecycle and Hang Prevention 13

10. Idle CPU, Message Races, and Resource Cleanup 13

11. User Generation Studio and Reliable Million-Row Output 14

12. Million-Row Recommendation Pipeline 15

13. Adaptive Multi-node Memory and Heap Controls 17

14. Fast Post-recommendation Persistence 19

15. Important Diagnoses 20

16. Verification Record 21

17. Consolidated Risk Register 22

18. Release and Research Recommendations 23

Appendix A. Change Ledger 24

Appendix B. Principal Source Files 25

Appendix C. Complexity Summary 25

Appendix D. Terminology 25

Appendix E. Final Assessment 26

1. Scope, Method, and Evidence

1.1 Scope

The scope is the complete development conversation represented in the local workspace: requested code changes, implementation decisions, later corrections, diagnoses that intentionally produced no code change, Git publication, and the current state of the repository. Generated experiment outputs are discussed as evidence but are not treated as source-code features.

The phrase “before” refers to the behavior present before the corresponding request, not necessarily one single historical commit. The project evolved through several intermediate states. For reproducibility, this report therefore identifies delivery state for each work item: committed and pushed, implemented only in the current workspace, or analysis-only.

1.2 Evidence Base

Evidence	Use in this report
Git history	Three commits between `origin/main` and current HEAD, including file-level statistics and dates.
Current source	Direct inspection of Java, Python, shell, and self-test implementations in the local working tree.
Conversation artifacts	User-supplied failure traces, screenshots, datasets, observed behavior, and requested invariants.
Generated corpus	`Reduced_94k_..._GENERATED.txt`: exactly 1,000,000 rows and 113,388,555 bytes.
Build verification	All production Java source compiled successfully with the project classpath.
Self-tests	Nineteen Java self-test classes compiled; all applicable tests passed with required arguments/runtime module flags.
Python verification	Doc2Vec modules syntax-checked and `doc2vec_streaming_self_test.py` passed.
Stress measurements	Million-row generation, ingestion, sparse SVM, and adaptive parallelism tests executed locally.

1.3 Verification Method

1. Compile all production Java files against `lib/*` into an isolated temporary classes directory.
2. Compile every Java self-test against those production classes.
3. Run ordinary tests, then rerun Weka tests with the same Java module-opening flags supplied by the application launcher.
4. Run million-row ingestion against the generated Reduced_94k corpus.
5. Run sparse SVM stress verification with two requested workers under a 4 GB heap.
6. Run the million-row user-generation stress test.
7. Syntax-check the active Python scripts and execute the streaming Doc2Vec self-test.
8. Render the report to pages, inspect every rendered page, and regenerate if layout defects are found.

1.4 Interpretation Limits

- Passing unit and stress tests does not prove every end-to-end GUI/JADE path under every dataset and operating system.
- The measured SVM speedup covers the stress-test model workload, not full text processing, JADE messaging, graph work, or all algorithms.
- Doc2Vec clean retraining is not guaranteed bit-for-bit deterministic because the existing model uses multi-worker Gensim training.
- The worktree contains runtime artifacts and uncommitted code; a future clean checkout of `origin/v3.2` will not contain most later changes.
- Common Neighbors remains outside the six working recommendation implementations and should not be described as fully supported.

2. Starting Architecture and Failure Surface

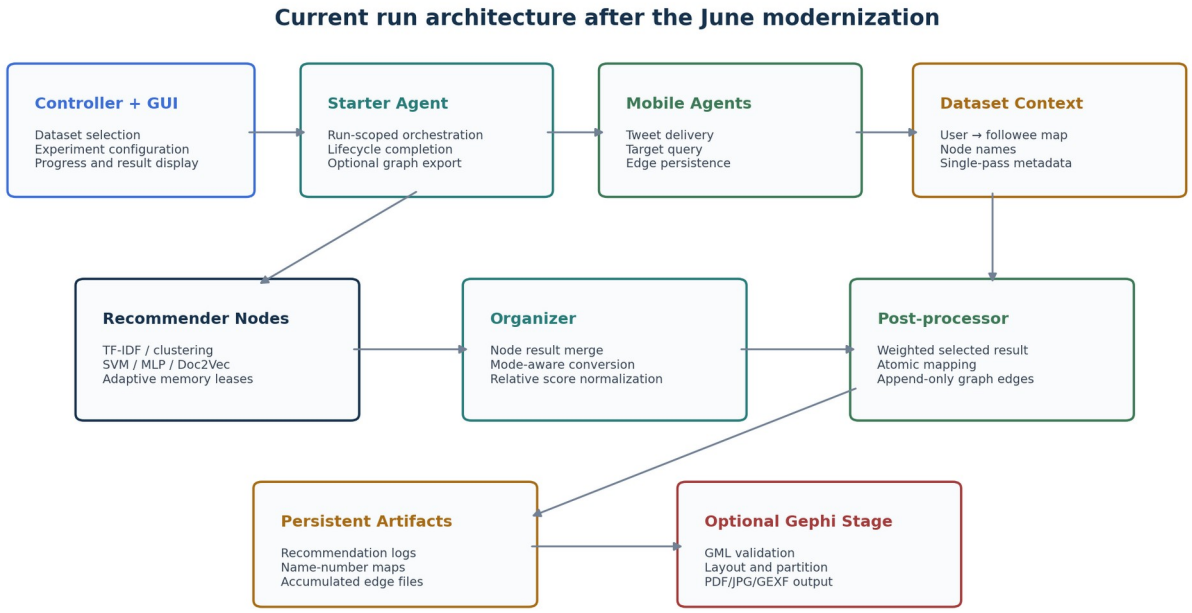


Figure 1. Current architecture after the June modernization. The design remains a JADE multi-agent pipeline, but configuration, run identity, dataset context, typed results, and post-processing are now explicit.

2.1 Core Execution Model

The simulator is a JADE multi-agent application. `ControllerAgent` loads a dataset, creates user and recommender agents, and transfers configuration. `StarterAgent` orchestrates a simulation. `MobileAgent` instances publish text and later query for recommendations. `RecommenderAgent` instances preprocess assigned text shards and run one of the algorithms. `OrganizingAgent` merges distributed scores. The recommendation is shown in `ControllerAgentGui`; edge artifacts are persisted and the optional Gephi stage constructs final graph outputs.

This architecture was viable for the original research datasets but accumulated several forms of hidden coupling: algorithm semantics were inferred from integer constants; graph generation was fused to successful completion; repeated runs reused unscoped messages; post-processing reread the original corpus; and the GUI exposed controls without a formal state model. Large generated datasets amplified every inefficiency.

2.2 Principal Pre-change Problems

Area	Pre-change behavior	Consequence
Doc2Vec batch	Only the first target was passed to Python and its score map was assigned to every selected target.	Silent correctness failure: different users appeared to receive independent results but did not.
Experiment isolation	Transient graph and recommendation files survived into the next dataset or run.	Old edges and outputs could contaminate a new experiment.
GUI	Dense legacy Swing form, fragile native dialogs, raw text recommendations, weak progress state.	Poor discoverability and several interaction defects, including disabled Initialize and dialog crashes.
Recommendation objective	Algorithms silently returned different entity types.	Doc2Vec/user rankers returned users while SVM/MLP returned field-0 accounts; users could easily compare unlike outputs.
Graph phase	Successful recommendation runs always entered Gephi visualization/export.	Unwanted latency and no way to defer graph rendering while continuing edge accumulation.

Area	Pre-change behavior	Consequence
Repeated runs	Long-lived agents could consume stale completion or score messages.	Runs could hang or complete against state from a previous simulation.
Idle agents	Several behaviors polled with short tick intervals.	High CPU persisted after computation, sometimes long after a graph was produced.
Generation	Legacy generator was monolithic, memory-heavy, weakly validated, and not atomic.	A million-row request could be partial, malformed, or operationally unreliable.
Model memory	Dense ARFF rows and unrestricted model concurrency multiplied 70k-feature structures per node.	SVM nodes crashed with `OutOfMemoryError`; surviving results were incomplete.
Post-processing	Each target rescanned the corpus repeatedly and rewrote a full Dataset-Test file.	A result appeared in the GUI but completion lagged for minutes on a million-row corpus.

2.3 Design Invariants

The implementation work was governed by the user's repeated requirement that the program's purpose and existing recommendations not be broken. The practical interpretation was:

- Algorithm Default must retain the native algorithm and downstream merge behavior.
- Batching must compute each target independently, not reinterpret one target's score map.
- Performance work may change representation and scheduling but not the mathematical score inputs or merge formula.
- Disabling graph output may skip visualization and export, but it must not discard recommendation edges or accumulated graph state.
- Reset must remove experiment-derived state but preserve input datasets, archived Results, and reusable model caches unless explicitly requested.
- Failures must not be converted into plausible-looking fake recommendations.

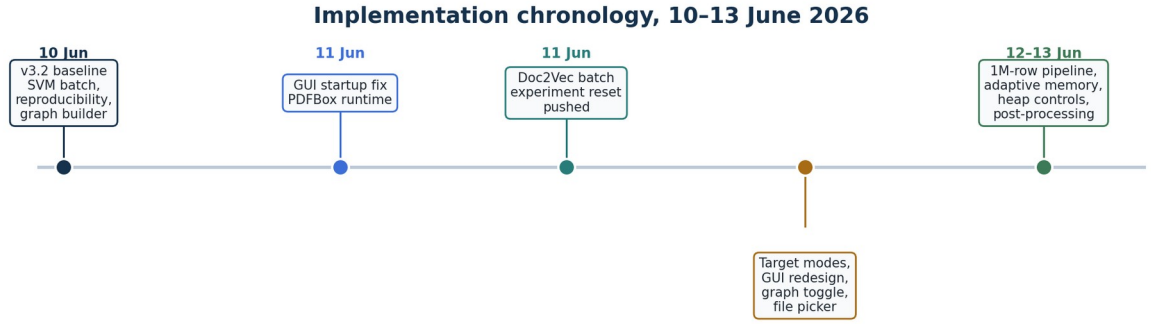


Figure 2. Implementation chronology. The first three milestones are Git commits; later milestones are present in the current local working tree.

3. Git History and Delivery State

Commit	Date	Subject	Scope	Delivery
`fe57855`	10 Jun 2026	Add v3.2 simulator updates	SVM batch selector, reproducibility support, graph builder, shared Doc2Vec infrastructure, broad agent updates.	Committed and pushed
`6a00b58`	11 Jun 2026	Fixed GUI not opening	Startup script correction and bundled PDFBox application JAR.	Committed and pushed
`cb47942`	11 Jun 2026	Add Doc2Vec batch mode and experiment reset	Correct multi-target Doc2Vec, reset-file service, GUI/controller wiring, tests.	Committed and pushed

Commit	Date	Subject	Scope	Delivery
Working tree after `cb47942`	11–13 Jun 2026	Uncommitted modernization	GUI redesign, target modes, graph toggle, lifecycle, resource fixes, 1M generation and ingestion, adaptive memory, heap UI, post-processing.	Local workspace only

The committed branch is `v3.2` tracking `origin/v3.2`; its current HEAD is `cb47942`. The remote is `https://github.com/aabhari/test.git`. The three commits between `origin/main` and HEAD contain 20 changed files, 4,796 insertions, and 2,113 deletions. The current uncommitted source diff is much larger: the existing tracked source changes alone include more than ten thousand insertions and almost four thousand deletions, plus twenty-three new production classes and fourteen new Java tests.

> **RELEASE IMPLICATION** Cloning or resetting to `origin/v3.2` reproduces Doc2Vec batch and the first reset implementation, but it does not reproduce the complete system described in Chapters 6–15. A clean, reviewed commit series is required before the later work is portable.

3.1 Non-source Artifacts in the Worktree

The working tree also contains generated corpora, ARFF files, Doc2Vec caches, recommendation logs, timing logs, graph-edge files, `.DS_Store` files, compiled `classes/`, Python bytecode, and experiment outputs. These are not all intentional source changes. `lib/gephi-toolkit.jar` also appears modified because the checked-out large binary and Git/LFS representation differ; its content was confirmed locally as a 67,946,596-byte ZIP/JAR, but it should not be staged casually.

- Generated datasets and Results files are evidence and experiment output, not implementation source.
- Doc2Vec shared cache files are performance artifacts and should be governed by `.gitignore` or deliberate release policy.
- The final code commit should be prepared from an explicit allow-list of source, tests, scripts, and documentation.
- A clean checkout build must be performed after committing to prove that no untracked helper class is accidentally required.

4. Correct Doc2Vec Batch Mode

> **DELIVERY STATE** Committed and pushed in `cb47942`, then further hardened in the local working tree.

4.1 Before

The Java Doc2Vec branch assembled a list of selected users but invoked the Python path using only `usersRec.get(0)`. After Python returned a score map for that first target, Java assigned the same map to every selected target. Enabling the existing SVM batch field for Doc2Vec would therefore have produced silently false output: the GUI would list many target users, but every target would inherit the first user's ranking.

4.2 Implementation

`Doc2VecBatchRunner` now writes the selected target names to a temporary UTF-8 file and starts one Python process for each recommender node. `doc2vec_batch.py` loads the shared model/index once in that process, loops through every target, and calls the same `score_candidate_shard()` function used by the active single-user design. Each node evaluates only its assigned candidate shard, preserving the distributed partition rule.

The interprocess protocol is structured rather than dependent on human-readable console text. Records are prefixed with `DOC2VEC_SCORE`, `DOC2VEC_ERROR`, or `DOC2VEC_DONE`; user, candidate, and error fields are Base64 encoded and tab delimited. Java constructs an independent score map per target and reports explicit per-user completion or failure.

Step	Current behavior
Model preparation	A shared Doc2Vec model is trained or reused under a cache signature that includes dataset path, size, modification time, parameters, workers, and script version.
Memory sharing	Normalized document vectors are stored in NumPy format and loaded with `mmap_mode="r"`, reducing duplicate resident model memory across Python processes.
Batch scoring	Every requested target is scored independently against each node's candidate shard.
Node merge	The organizer receives per-target maps and merges shards using the existing downstream workflow.
Failure	A failed or missing target produces an explicit error/empty result rather than a fabricated `-1.0` candidate map.

4.3 Recommendation Preservation

For a fixed cached model, a user scored in batch receives the same candidate-shard calculation as that user scored alone because the exact same scoring function and arguments are used. Batching changes process reuse and result organization, not the similarity calculation. Candidate sharding and organizer merging remain intact.

The qualified limitation is model training. Gensim Doc2Vec uses multiple workers in the retained configuration (`dm=0`, `dbow_words=1`, vector size 300, 20 epochs, `min_count=4`, negative sampling 3). Clean retraining can vary slightly because multi-worker training is not bit-for-bit deterministic. Batch mode does not create that limitation. Forcing single-worker deterministic training would alter runtime and could change the established model, so it was not silently imposed.

4.4 Efficiency and Risks

- Benefit: one Python process per node loads the scoring index once for all selected targets, avoiding per-user interpreter and model-load overhead.
- Benefit: memory-mapped vectors and one-thread BLAS defaults reduce Python process multiplication and CPU oversubscription.
- Cost: scoring remains approximately linear in target count times candidate count; batching removes overhead but does not make the underlying similarity search sublinear.
- Risk: a target absent from the Doc2Vec tag index cannot be scored and must remain an explicit failure/no-candidate case.
- Risk: each recommender node is still a separate Python process; operating-system memory and file-cache behavior are outside the Java heap coordinator.
- Risk: cache validity depends on the signature inputs and filesystem metadata being accurate.

5. Brand-New Experiment Reset

> **DELIVERY STATE** The first reset implementation is committed and pushed; later target-mode and dataset-specific cleanup is local workspace work.

5.1 Before

Graph, recommendation, timing, and helper files persisted between runs. A new text dataset could therefore inherit old edge files, old generated GML, prior recommendation lists, or temporary output-choice files. An earlier cleanup was tied to dataset loading/Get Users, which made a routine action destructive and obscured the boundary between continuing an experiment and starting a new one.

5.2 Implementation

A dedicated Reset Experiment command now performs two coordinated actions. First, `ControllerAgent` terminates active agent controllers and work/user-simulation containers, replaces the in-memory database, clears maps, lists, timers, user selection, and configuration, and returns the GUI to its initial state. Second, `ExperimentResetFiles` deletes known transient outputs.

Deleted	Preserved
Current graph exports such as GEXF, layout JPG, and Gephi PDF outputs.	Original source datasets.
Temporary output-choice, text-file-name, separator, and tweet-count files.	Archived `Results/` history unless explicitly overwritten by normal execution.
Default, Similar Users, and Followee Accounts recommendation-list helper files.	Doc2Vec model/vector caches, because they are reusable computational artifacts.
Selected-dataset `Dataset-Test-*`, edge, and GML files, including mode-specific suffixes.	Unrelated datasets' graph state and name-number files.

The GUI presents a confirmation warning because reset intentionally destroys current experiment state. The cleanup is no longer coupled to Get Users. A `resetPendingDatasetSelection` flag ensures that, after reset, the first selected dataset also receives its known dataset-specific stale-output cleanup.

5.3 Risks

- The reset service can only delete filenames and suffixes it knows about; new artifact types require the reset inventory to be updated.
- A permission or file-lock failure is reported but can leave partial contamination on disk.
- Preserving caches is intentional, but a cache bug or incorrectly reused external cache would not be cured by Reset Experiment.
- The reset is a workflow boundary, not a database transaction; external tools writing to the same files remain outside its control.

6. GUI Modernization and Interaction Repairs

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed after `cb47942`.

6.1 Structural Redesign

The legacy Swing window was reorganized into a three-column workspace: Data Source and users on the left, Experiment Setup in the center, and Live Output on the right. The application title is aligned to the far left, the redundant SVM/Doc2Vec/Batch/Graph badges below it were removed, and global actions—File, User Studio, Reset Experiment, Quit—are grouped in the header.

Experiment Setup now reflects conceptual order rather than field history: Model & Objective, Workload, Runtime Resources, Data Window, Text Processing, Post-processing, and Run Pipeline. Numeric settings use stepper controls, modes use selectors, binary options use checkboxes, and algorithm capability badges show whether the native output is users or accounts and whether batching is available.

Surface	Change	Operational value
Data source	Four selectable source cards—Text file, Database, Generated, Artificial—with a selected-source detail panel.	Makes source mode visible and prevents the prior menu/button ambiguity.

Surface	Change	Operational value
Experiment setup	Grouped controls, numeric steppers, algorithm/objective summary, and validation-aware enablement.	Reduces configuration errors and makes algorithm consequences legible.
Run controls	Initialize and Run Simulation have explicit state transitions.	Prevents launching before users/configuration exist and clarifies the two-stage workflow.
Live output	Tabs for timing, previous output, and recommendations.	Separates diagnostic logs from user-facing results.
Status bar	“Current progress” appears before a colored stage pill and human-readable activity.	Shows PROCESSING, FEATURES, SCORING, MERGING, DELIVERING, and GRAPH phases.
Visual system	Consistent restrained surfaces, icons, spacing, typography, hover transitions, and compact cards.	Improves scanability without changing JADE or model behavior.

6.2 Initialize Button Repair

The modernized GUI initially left Initialize disabled after users had been loaded. The repaired callback now enables it when the returned agent list is non-empty: `initializeButton.setEnabled(!agentsList.isEmpty())`. Reset, initialize, run, and source changes continue to disable or re-enable controls according to state.

6.3 Recommendation Result Cards

Raw monospaced recommendation blocks were replaced with structured cards. Each target card shows an avatar, target name, result count, ranked candidates, rank colors, candidate details, and score. The legacy text buffer remains available for diagnostics and prior output, while typed `RecommendationResultPayload` objects drive the user-facing view.

One residual wording issue remains: the card labels values as MODEL SCORE even though several paths display a max-normalized relative rank. The value can be technically correct yet semantically misread as probability or confidence. A future release should label it Relative Score unless the score kind is explicitly probabilistic.

6.4 Dialog Crash and Modern File Picker

The native `JFileChooser` path repeatedly failed on macOS with `IllegalComponentStateException: The dialog is decorated` while Swing attempted to set a dialog background during construction. Instead of continuing to patch global look-and-feel behavior around native dialogs, a custom `ModernFilePicker` was added.

- Asynchronous directory loading and dataset inspection using `SwingWorker`.
- Debounced 280 ms search within the current folder tree.
- Search safety caps of 20,000 inspected entries and 1,000 displayed results.
- Recent, project, Dataset, TestedDataset, home, Downloads, Desktop, Documents, and filesystem-root shortcuts.
- Back, up, home, refresh, direct path entry, file filters, hidden-file toggle, double-click selection, keyboard shortcuts, and drag-and-drop.
- Persistent recent files and last directory via Java Preferences.
- Dataset compatibility preview before selection.

6.5 GUI Risks

- `ControllerAgentGui.java` is now approximately 5,623 lines; future work should extract stable view components without changing behavior.
- Custom-painted Swing components require strict Event Dispatch Thread discipline; automated pixel and accessibility testing is still limited.
- Search is intentionally capped and may not enumerate every file in a very large tree.
- Preferences and drag-and-drop behavior vary by operating system and security policy.
- The new interface is visually modern but remains a desktop Swing application, not a native Apple UI framework.

7. Recommendation Target Modes

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed.

7.1 Why the Feature Was Needed

The six working algorithms did not all answer the same question. Similarity, K-Means, K-Means Euclidean, and Doc2Vec ranked other user identities. SVM and MLP classified the account stored in field 0—the reference/followee account. This explained why a retail dataset could show retailer names under SVM but person names under Doc2Vec. The behavior predated these modifications; it was not caused by batch mode.

A new `RecommendationTargetMode` makes the desired output explicit: Algorithm Default, Similar Users, or Followee Accounts. The default path bypasses conversion and preserves each algorithm's historical behavior. Explicit cross-type modes use typed score payloads and deterministic merge logic.

7.2 Mode Semantics by Algorithm

Algorithm	Algorithm Default	Similar Users	Followee Accounts	Verdict
Similarity	Ranks users by legacy TF-IDF cosine similarity.	Same native user ranking.	Groups candidates by their field-0 account and keeps the best matching user score per account.	Correct as native + explicit aggregation.
K-Means	Ranks users using the legacy cluster/similarity path.	Same native user output.	Maps ranked users to accounts, retaining the best representative score.	Correct within the legacy K-Means definition.
K-Means Euclidean	Ranks users with the existing Euclidean cluster path.	Same native user output.	Maps user scores to field-0 accounts using maximum evidence.	Works; inherits legacy asymmetric sparse-distance semantics.
Doc2Vec	Ranks users by document-vector similarity.	Same native user ranking.	Maps similar users to their accounts; card detail identifies the best matching user.	Correct conversion; does not retrain Doc2Vec as an account classifier.
SVM	Predicts/ranks field-0 followee classes.	Scores candidate users using class signal multiplied by sparse text cosine evidence.	Same native followee classification.	Default/account mode is native; user mode is a documented hybrid.
MLP	Predicts/ranks field-0 followee classes.	Uses merged classifier signal multiplied by candidate text-similarity evidence.	Same native followee classification.	Default/account mode is native; user mode is a documented hybrid.
Common Neighbors	Not a complete active recommendation implementation.	Unsupported.	Unsupported.	Do not present as production-ready.

7.3 User-ranker to Followee Conversion

For a user-ranking algorithm, every candidate user is associated with its field-0 account through `RecommendationDatasetContext`. Candidates sharing the same account are collapsed. The maximum candidate-user score is retained as the account score, and evidence records the best-matching user. MAX is used because summing would favor accounts merely because more candidate users happen to belong to them.

7.4 Classifier to Similar-user Conversion

For SVM or MLP, the merged classifier distribution provides account-level signal. A candidate user receives a hybrid score equal to its account-class share multiplied by sparse text cosine similarity to the target. This ensures that the classifier's learned class preference still matters while selecting a concrete user. The conversion is useful, but it is not equivalent to training a native pairwise user recommender.

7.5 Validation and Isolation

- Explicit modes are limited to text datasets because they require the six-field user/account relationship.
- The target must exist and sufficient users/classes must remain after holdout.
- Classifier conversions require at least two training classes after excluding the target.
- Converted outputs use mode-specific recommendation, edge, and GML suffixes to avoid mixing unlike semantics.
- Payload statuses distinguish OK, NO_CANDIDATES, UNSUPPORTED, and FAILED.
- Tie ordering is deterministic by candidate name after score comparison.

7.6 Score Normalization

> **WHY THE FIRST RESULT IS 100** Both the legacy organizer and the new merger divide every candidate by the maximum score and multiply by 100. The top item is therefore always 100 whenever the maximum is positive. A second SVM score of 0.5 means approximately 0.5% of the top item's merged score after normalization; it is not automatically a 0.5% probability.

7.7 Risks

- Converted modes intentionally change recommendation objective; only Algorithm Default is the strict compatibility path.
- A max-normalized score is relative to candidates in that run and should not be compared as calibrated confidence across algorithms or datasets.
- Field 0 must genuinely represent an account/followee category; malformed retail or Twitter corpora can create semantically meaningless conversions even when parsing succeeds.
- Current-followee exclusion is not automatically equivalent across all native and converted modes.

8. Optional Graph Output with Persistent Edge State

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed.

8.1 Before

Recommendation completion and final graph visualization were fused. Even when the user only wanted recommendations, Starter entered the Gephi build, layout, partition, PDF/JPG, and graph-export phase.

8.2 Current Behavior

A Generate Final Graph Output checkbox is serialized in `RecommendationExperimentConfig` and passed through Controller to Starter. If checked, behavior remains the legacy-compatible graph build/export/open path. If unchecked, Starter skips Gephi visualization and export and completes after recommendation querying and edge persistence.

> **CRITICAL INVARIANT** Unchecked does not mean "discard graph data." MobileAgent still appends original and selected recommendation edges. If ten runs are executed with graph output disabled and the eleventh is executed with it enabled, the final graph includes accumulated edges from the earlier ten runs, provided the experiment was not reset.

Mode-specific edge and GML names prevent Similar Users and Followee Accounts experiments from being silently combined. The graph builder validates generated GML, and the embedded Gephi project is closed in a `finally` path to release memory and background resources.

8.3 Risks

- Accumulation is deliberate; users must press Reset Experiment before a logically new experiment.
- Skipping Gephi reduces export time but does not skip model computation or required edge persistence.
- If an edge append fails, recommendation calculation may still be valid while the future graph is incomplete; the GUI logs this distinction.
- Graph communities depend on the semantics and diversity of accumulated edges, not merely on whether batch mode was used.

9. Repeated-run Lifecycle and Hang Prevention

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed.

Repeated simulations reused long-lived JADE agents. Generic conversation IDs and per-run fields could allow stale text-processing, score, merge, or completion messages to enter a later run. A run could remain disabled in the GUI, wait forever for a dead agent, or complete with state that did not belong to the active request.

`RecommendationRunLifecycle` now creates a UUID-backed `StartSim-*` identifier, classifies messages as a new run, current run, or retired run, and resets per-run state in Recommender, Organizer, and Mobile agents. The lifecycle retains the sixteen most recent retired IDs and guards text-processing completion so one run cannot double-complete.

`StarterAgent.finishSimulationLifecycle()` centralizes completion and is idempotent. Normal, graph-disabled, error, and timeout-adjacent paths converge on the same cleanup and `doDelete()` behavior. This specifically addresses the "recommendations visible but simulation never finishes" class of defect.

- Compatibility: legacy messages normalize to a legacy run identifier.
- Risk: only sixteen retired IDs are retained; exceptionally delayed traffic older than that window is not remembered indefinitely.
- Risk: external or older agents that omit run IDs remain less isolated than fully updated agents.
- Performance effect: negligible metadata and comparisons relative to model work; correctness benefit is substantial.

10. Idle CPU, Message Races, and Resource Cleanup

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed.

10.1 Observed Failure

The application could remain near 1,200% CPU long after a simulation and graph had finished. The problem was not limited to the redesigned GUI. Several JADE behaviors continued polling for messages with short intervals or unfiltered receives, so idle agents repeatedly woke across many cores. Gephi project state could also remain open.

10.2 Changes

Change	Effect on behavior	Effect on recommendations
Mobile message delivery changed from short ticker polling to blocking cyclic receive.	No CPU wake-up until a matching JADE message arrives.	None; same messages and handlers.
Idle recommendation-query handlers call `block()` when no request exists.	Stops busy waiting between user queries.	None.
Recommender agents block when no work message exists.	Idle nodes cease spinning after computation.	None.

Change	Effect on behavior	Effect on recommendations
Score handlers use specific MessageTemplates.	The intended behavior consumes the score; unrelated handlers cannot steal it.	Reduces a pre-existing race; does not alter score math.
Completion popup is scheduled asynchronously on Swing EDT.	Agent cleanup proceeds without waiting for a modal UI action.	None.
Gephi project is closed after export.	Releases graph resources and background state.	None.

10.3 Performance and Risk

Blocking receive changes scheduling, not model computation. The only practical timing difference is negligible wake-up and queue-delivery latency when a new message arrives. CPU and battery use should drop sharply while the GUI is open but idle or after a simulation completes.

- A handler that uses an incorrect MessageTemplate can now wait indefinitely instead of accidentally consuming a broad message; the templates therefore require regression tests.
- The change does not eliminate CPU used by an active model, Python process, Gephi layout, or deliberate tweet-delay simulation.
- JADE containers and agents still need orderly termination; blocking alone does not correct a leaked live agent.

11. User Generation Studio and Reliable Million-Row Output

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed.

11.1 Before

The original `UserGenSimAgent` contained roughly two thousand lines of generation, statistics, clustering, file I/O, and GUI interaction. It loaded and transformed corpus structures with weak input validation, printed large diagnostics, and wrote directly to the intended output. A crash or malformed source could leave a partial file that looked complete. Exact row-count guarantees were not strong enough for a one-million-row request.

11.2 New Engine

Generation logic now lives in `UserGenerationEngine`; `UserGenSimAgent` is a thin agent wrapper that validates arguments, invokes the engine, and forwards staged progress to the User Studio GUI. `DatasetRow` centralizes strict six-field parsing for both generation and ingestion.

Stage	Current implementation
Validation	Every nonblank row must contain six tab-separated fields with parseable numeric IDs. A user is expected to map to one reference account; conflicts are warned and the first mapping is retained.
Preprocessing	Retweets and records with fewer than three usable terms are filtered according to generation rules.
Modeling	Per-user sparse TF-IDF term distributions are built; Weibull parameters use maximum-likelihood estimation with regression and fallback paths.
Allocation	`allocateExactly()` uses proportional largest-remainder allocation so the sum equals the requested target exactly.
Streaming output	Rows are written with a 128 KiB buffer to a hidden UUID `.part` file rather than accumulated in a giant output list.
Durability	The file descriptor is synchronized, output row count is verified, and an atomic move publishes the final file. Failure deletes the temporary file.
Capacity	Estimated output size is compared with usable disk space while preserving at least 64 MB free.

Stage	Current implementation
Validation analytics	Sparse user vectors and a bounded three-cluster homophily summary are produced for the generated corpus.

11.3 Correctness Properties

- Requested positive row counts are exact; zero means generate the usable source-row count.
- Counts above `Integer.MAX_VALUE` are rejected because internal identifiers and downstream structures are not designed beyond that limit.
- A deterministic seed can reproduce output; the normal GUI path may use a random seed.
- The output retains the required six-field tab-separated format and valid generated tweet IDs.
- Mixed line endings, malformed input, smaller-than-source targets, failure cleanup, deterministic generation, and million-row output are covered by self-tests.

11.4 Measured Result

The generated Reduced_94k corpus present in the workspace contains exactly 1,000,000 lines and is 113,388,555 bytes. The dedicated million-row generation self-test completed successfully in 878 ms on its bounded synthetic fixture. That elapsed time should not be treated as the expected runtime for a full 94,000-row source model; it verifies exact streaming publication and cleanup mechanics.

11.5 Limits

- The source corpus and per-user generation models still occupy memory; output is streamed, but source-model memory scales with source vocabulary and user count.
- Three validation centroids are dense across the vocabulary, a bounded but nonzero memory cost.
- Generated text is a synthetic bag-of-terms representation guided by TF-IDF/Weibull sampling; it is not intended to reproduce natural language discourse.
- A corpus in which one user maps to multiple field-0 accounts is semantically ambiguous even if the first mapping rule permits generation.
- A valid million-row file is not proof that every legacy recommendation algorithm can process it efficiently; the next chapter addresses that separate concern.

12. Million-Row Recommendation Pipeline

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed.

12.1 Original Failure

The first million-row SVM run failed even though the generated file was valid. Eight recommender nodes shared one Java process capped at approximately 1,424 MB. Each node constructed a roughly 70,000–74,000-feature ARFF representation and several Weka models. Nodes 3, 4, 5, and 7 threw `OutOfMemoryError`; dead nodes could not report completion, so surviving recommendations were incomplete and the workflow could remain stuck.

The root problem was architectural multiplication, not a malformed dataset: dense feature serialization, repeated scans, per-node shared-data copies, and unrestricted concurrent model construction.

12.2 Streaming Dataset Ingestion

`ControllerAgent` now reads text datasets through a buffered UTF-8 stream and `DatasetRow.parse()`. During that single pass it builds the user-to-followee map, reference-user set, graph node names, and—only when required—controller-side

document aggregation. Malformed rows are skipped with a warning instead of crashing later inside algorithm-specific parsing.

`InMemoryDb` adds a `tweetsByUser` index. Previously, repeated `getTweetsFromUser()` and count operations scanned the entire tweet list. They now retrieve an indexed per-user list in approximately O(1 + k), where k is that user's tweet count, after a single index build.

12.3 Sparse Feature Representation

`SparseArffWriter` emits Weka sparse rows such as `{featureIndex value, classIndex classValue}`. The ARFF header must still enumerate the vocabulary, but each user row stores only nonzero features instead of tens of thousands of comma-separated zeros. The Weka self-test confirms that sparse and dense representations preserve instance values and model scores.

`Point` now treats absent terms as zero when comparing a sparse point to a centroid, eliminating the need to materialize full zero-filled maps. This preserves the legacy distance routine's iteration domain; it does not redefine the Euclidean algorithm as a new symmetric full-vocabulary metric.

12.4 Algorithm-specific Memory Work

Algorithm/path	Memory change	Recommendation impact
SVM	Sparse ARFF rows, staged model admission, temporary-map cleanup.	Same Weka SMO input values and seeded behavior; representation changes, not intended scoring.
MLP	`MlpModelAverager` loads one model at a time and accumulates connection weights.	The legacy averaging formula is preserved deliberately, even though it is not rewritten as a new statistical average.
Doc2Vec	Streaming tagged-document corpus; normalized vector cache; NumPy memory mapping; BLAS threads default to one.	Same model parameters and candidate similarity function for a fixed cache.
K-Means	Sparse point access and earlier release of raw tweet structures.	Legacy cluster/scoring logic retained.
Similarity	Benefits from streaming aggregation and indexed data access.	Cosine calculation retained.
Common Neighbors	Not promoted to a supported million-row path.	No claim of correctness or scalability.

12.5 Cleanup of Intermediate Structures

After text aggregation and feature extraction, large raw tweet maps, temporary documents, and compatibility files are cleared when no longer needed. The controller does not generate Doc2Vec or TF-IDF compatibility artifacts for algorithms that do not consume them. This reduces peak live-set overlap, which matters more than final steady-state memory.

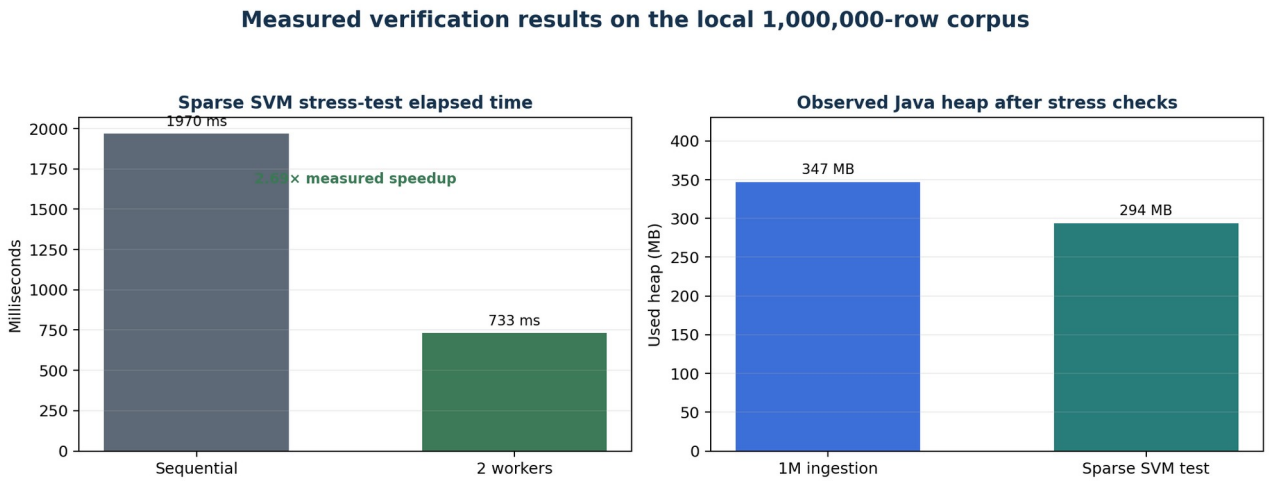


Figure 3. Local stress-test measurements on the 1,000,000-row Reduced_94k-generated corpus. These isolate ingestion and sparse SVM model work rather than measuring the entire JADE simulation.

12.6 Measured Verification

Check	Dataset/workload	Observed result
Generation	Synthetic million-row self-test	Passed; exactly 1,000,000 rows; 878 ms on bounded fixture.
Ingestion	Reduced_94k-generated corpus	1,000,000 rows, 40 users, all rows assigned; approximately 347 MB used Java heap.
Sparse SVM	1,000,000 rows aggregated to 40 users, 74,618 terms, 4 classes	Passed with 2 workers; max concurrency 2; approximately 294 MB used heap.
Parallel comparison	Two sparse SVM model tasks	Sequential 1,970 ms; concurrent 733 ms; measured 2.69× speedup.
Doc2Vec streaming	Temporary streaming corpus/cache test	Model trained, shared cache saved, memory-mapped scoring test passed.

12.7 Remaining Scaling Limits

- Weka SVM training can still have quadratic or worse behavior in training-instance count depending on kernel and data; sparse storage does not remove algorithmic complexity.
- MLP topology still grows with input feature count and hidden-layer design; scheduling avoids simultaneous overload but cannot make an oversized network cheap.
- K-Means maintains dense centroid arrays for the vocabulary and may remain slow on very wide corpora.
- Doc2Vec Python memory is outside the Java heap budget; multiple node processes can still create operating-system pressure.
- The one-million-row tests contain only 40 aggregated users and four field-0 classes. A corpus with hundreds of thousands of users is a different workload.

13. Adaptive Multi-node Memory and Heap Controls

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed.

13.1 Why a Fixed Node Cap Was Insufficient

A crude protection such as “never run more than four model nodes at once” prevents some crashes but wastes capable hardware and makes node count misleading. Conversely, blindly launching every requested node repeats the original out-of-memory failure. The required behavior is conditional: run all nodes concurrently when their estimated working sets fit; otherwise run safe waves.

13.2 RecommendationMemoryCoordinator

The coordinator implements a fair lock/condition admission system. It computes a model budget from the current maximum Java heap after reserving the larger of 256 MB or 35% for JADE, GUI, text data, and general runtime state. A live-heap safety margin is also retained. Each expensive stage requests a lease with a workload type and estimated bytes.

Workload	Estimate basis	Admission behavior
SVM	Feature count, ARFF size multiplier, training-instance kernel workspace; aligned and clamped to 96–384 MB.	Concurrent while total reservation, live heap, CPU, and requested-node limits permit.
MLP training	Connection topology and dense vector estimates; clamped to 144–512 MB.	Typically more conservative than SVM due to large object graphs.
MLP average/score	Model-file size multiplier; clamped to 96–384 MB.	Prevents multiple large Neuroph models from loading together when heap is tight.
Generic	Default bounded reservation.	Used for expensive stages without a specialized estimator.

The GUI status listener reports active and waiting model stages, reserved memory, model budget, and requested nodes. The scheduler therefore makes a reduced parallel wave visible instead of silently pretending that all requested nodes are concurrent.

13.3 Heap Configuration and Restart

`RuntimeResourceSettings` detects current max heap and, where supported, physical memory. The GUI exposes Maximum Java heap (MB) in 512 MB steps. Apply & Restart writes `.simulator-runtime.properties`, warns above approximately 75% of physical memory, and relaunches the application with `-Xms64m -Xmx{MB}m` through `RuntimeRelauncher` and `RuntimeRelaunchHelper`.

The macOS/Linux startup scripts read the same property and retain a 1,424 MB default when no setting exists. Java 9+ module-opening flags remain in the launcher so Weka and other older dependencies can operate.

13.4 What Changes for the User

Situation	Before	Now
Enough heap for all requested nodes	All nodes ran, but without an explicit safety proof.	All requested nodes can run together; scheduler records the reservation.
Not enough heap	Nodes could crash and leave incomplete merges.	Nodes wait and execute in safe waves; all requested work still runs.
Larger configured heap	Required editing a script or environment variable.	User sets heap in GUI and restarts.
Excessive heap request	No guidance.	GUI warns when request exceeds 75% of detected physical memory.

13.5 Risks and Interpretation

- Reservations are heuristics with clamps; an unusual Weka/Neuroph model can use more or less memory than estimated.

- Java heap size does not include Python Doc2Vec processes, native BLAS allocations, memory-mapped file cache, or Gephi native/graphics resources.
- A very large heap can reduce Java out-of-memory risk while increasing operating-system swapping and whole-machine instability.
- Changing concurrency changes wall-clock timing and interleaving, but the scheduler does not alter model parameters or score formulas.
- Restart necessarily discards the current in-memory experiment; the UI confirms before relaunch.
- Windows startup remains less polished than the modern macOS/Linux path and should receive a clean-checkout verification before release.

14. Fast Post-recommendation Persistence

> **DELIVERY STATE** Implemented in the current local working tree; not yet committed.

14.1 Before

The GUI displayed a recommendation before the simulation was complete. MobileAgent then scanned the selected dataset up to five times for recommended candidates, twice more for the target and original followee, rewrote the entire corpus into `important-stuff/Dataset-Test-*`, generated or reloaded mappings, and appended edges. On a million-row, approximately 107 MB corpus, a single target could trigger seven to nine million line reads plus a complete file rewrite. Batch runs repeated this work sequentially for every target.

The expensive Dataset-Test file was legacy output and was not consumed by the active recommendation or graph pipeline. The graph-disabled checkbox did not help because edge persistence correctly occurs before the optional Gephi phase.

14.2 Current Implementation

`RecommendationDatasetContext`, built during the controller's initial streaming read, already contains the identities, user-to-followee relationships, and graph node names needed after scoring. `RecommendationPostProcessor` uses that context to select the persisted recommendation, ensure a name-number map, and append the original and new recommendation edges.

- No Dataset-Test corpus rewrite occurs in the active path.
- The old implementation remains behind `ENABLE_LEGACY_CORPUS_REWRITE` for forensic compatibility but is disabled.
- A single fallback scan occurs only if required identities or graph node names are absent from context.
- Name-number mappings are written atomically and cached by canonical path, modification time, and file length.
- Mapping creation and edge append use synchronization to prevent concurrent corruption.
- The legacy weighted random selection of which top recommendation becomes the persisted edge is retained.
- An edge-persistence failure is reported as a post-processing warning and does not falsely invalidate an already calculated recommendation.

14.3 Complexity Change

Operation	Before	Now
Candidate identity lookup	O(k × N) corpus scans, with k up to five recommendations.	O(k) map lookup from dataset context.
Target/followee lookup	Additional O(N) scans.	O(1) context lookup.
Dataset-Test output	O(N) read and O(N) write per target.	Disabled; no active cost.
Name-number map	Potential repeated load/build.	Atomic create and metadata-keyed cache.
Edge append	Small append after expensive scans.	Small synchronized append remains.

Observed stage logs after the change reported recommendation-choice and graph-edge persistence in approximately 0–1 ms for runs where the context and mapping were already available. That is the expected order of magnitude for small map lookups and append-only I/O, but cold mapping creation and filesystem latency can be higher.

14.4 Risks

- Correctness depends on the dataset context being built from the same selected dataset used for the run.
- The metadata cache can miss an external file edit that preserves both file length and modification timestamp, although that is unusual.
- Synchronized edge writes serialize a very small section by design; this protects file integrity at negligible cost relative to model work.
- Weighted follow selection remains intentionally random, so repeated runs can persist different edges even when ranked scores are identical.

15. Important Diagnoses That Did Not Require Algorithm Changes

> **STATUS** These findings explain observed behavior. They are not all code changes and should not be misreported as fixed model defects.

15.1 Why the Top Score Is Always 100

The organizer and new merger normalize by the largest positive score. Therefore the first item is 100 by definition. This applies to Doc2Vec, SVM, and other algorithms after merge. Very small second and third SVM values indicate an extremely dominant top class relative to the alternatives after normalization. The GUI should describe these as relative ranking values.

15.2 Why Doc2Vec Shows People and SVM Shows Retailers

Doc2Vec's native candidate tags are user identities. SVM's class label is field 0 of the six-field dataset. In Retail50K that field contains retailer/account names, so SVM returns retailers. This difference was present before the modifications and was the reason for asking the professor to clarify intended research semantics. The email draft was a communication artifact, not a source-code change.

15.3 Generated Twitter SVM Recommendations

The one-million-row generated Twitter corpus contains 40 users distributed among four field-0 classes: RyersonU, TheCatTweeting, weathernetwork, and TorontoStar. In Algorithm Default, SVM is effectively classifying which known account each user belongs to. It is therefore expected that many selected TorontoStar users receive TorontoStar as the top result and many weather users receive weathernetwork.

This is technically consistent with the existing classification objective but can be semantically unhelpful as a "new account to follow" recommendation because it often predicts the account already present in the row. The implementation was not silently changed after this diagnosis; the new target-mode framework gives the user a clearer choice of objective.

15.4 Two Communities Instead of Approximately Fifteen

A low Gephi community count is not sufficient evidence of a Doc2Vec batch defect. Community count depends on the edge-generation rule, persisted random choice, target coverage, repeated recommendations, recommendation objective, accumulated prior edges, and Gephi modularity settings. If many users receive the same few hubs, the graph naturally collapses into a small number of communities.

The correct investigative sequence is to verify per-target score diversity, confirm that each batch user has an independent map, inspect the appended edge distribution, reset stale experiment outputs, record the exact modularity

resolution/randomization settings, and compare against 618 equivalent single-user runs using the same cached model and edge-selection seed. Batch-score equality alone does not guarantee an identical final graph because the persisted follow edge is weighted-random.

15.5 Analysis of Twitter versus Retail Corpora

- Both dataset families can satisfy the six-field parser, but field meanings differ; syntactic validity is not semantic equivalence.
- A retail row commonly encodes retailer/account in field 0 and shopper/user in field 4, so followee classification has an intuitive product-domain label.
- A tested Twitter corpus often encodes a followed/reference account in field 0 and author identity in field 4.
- Similarity/user modes rely primarily on text and user identity; followee modes rely on field-0 class quality and class diversity.
- A corpus with one class cannot support meaningful SVM/MLP followee classification after target holdout.

16. Verification Record

16.1 Production Build

All production Java sources in `TwitterGatherDataFollowers/userRyersonU/*.java` compiled successfully against `lib/*` into an isolated temporary output directory. The compiler reported only existing deprecated/unchecked API notes.

The repository's `build.py` was not used for final compilation because it hard-stops under Python 3.12 and accepts only Python 3.9 or 3.10. Direct `javac` compilation therefore provided a cleaner verification in the current environment. This Python version restriction is a residual build-tool usability issue.

16.2 Java Self-tests

Test	Purpose	Result
AdaptiveModelConcurrencySelfTest	Parallel admission, requested limit, oversized serialization.	Pass
BatchRecommendationSelectorSelfTest	Target selection and batch rules.	Pass
ControllerStreamingAggregationSelfTest	Single-pass controller aggregation behavior.	Pass
Doc2VecBatchRunnerSelfTest	Structured multi-user Python protocol parsing.	Pass
ExperimentResetFilesSelfTest	Transient deletion and preservation rules.	Pass
GraphGenerationSettingSelfTest	Graph checkbox configuration propagation.	Pass
GraphGmlBuilderSelfTest	GML build/validation behavior.	Pass
InMemoryDbSelfTest	Indexed per-user lookup and mutation consistency.	Pass
MillionRowIngestionSelfTest	1M-row ingestion and user assignment.	Pass with generated corpus
MillionRowSparseSvmSelfTest	74k-feature sparse ARFF and parallel model execution.	Pass, 2 workers
MlpModelAveragerSelfTest	Streaming legacy-compatible model combination.	Pass
RecommendationPostProcessorSelfTest	Fast context path, fallback, mapping, edges.	Pass
RecommendationRunLifecycleSelfTest	New/current/retired run admission.	Pass
RecommendationScoreMergerSelfTest	Typed merge, conversion, normalization, errors.	Pass
RuntimeResourceSettingsSelfTest	Heap persistence and bounds.	Pass
SparseArffWriterSelfTest	Sparse/dense equivalence and Weka scoring.	Pass with launcher module flags

Test	Purpose	Result
SparsePointDistanceSelfTest	Missing sparse terms treated as zero.	Pass
SvmReproducibilitySelfTest	Stable seeded Weka behavior.	Pass with launcher module flags
UserGenerationEngineSelfTest	Allocation, validation, cleanup, determinism, 1M output.	Pass

16.3 Python Verification

- `doc2vec_shared.py`, `doc2vec_batch.py`, and the streaming self-test passed Python syntax compilation.
- `doc2vec_streaming_self_test.py` trained a shared Doc2Vec model, saved the cache, loaded the scoring representation, and passed.

16.4 Runtime Compatibility Note

When Weka tests were first run with a bare Java command, Weka printed an `InaccessibleObjectException` while trying to reflect into `java.lang`. The tests continued, but they were rerun with the same Java 9+ `--add-opens` flags present in the project launchers and passed without the stack trace. The application must continue to launch with those flags on modern JDKs.

17. Consolidated Risk Register

Risk	Severity	Likelihood	Current control	Recommended next action
Later changes are uncommitted and mixed with generated artifacts.	Critical	Certain	This report identifies delivery state.	Create reviewed source-only commits from an allow-list; verify a clean clone.
Converted recommendation modes may be mistaken for native model outputs.	High	Medium	Typed mode, candidate type, detail text, separate files.	Use “hybrid” wording in GUI/help and research reporting.
Relative scores labeled MODEL SCORE may be read as probabilities.	High	High	Documentation explains normalization.	Rename to Relative Score and expose raw/score-kind metadata where useful.
Adaptive memory estimates can underpredict an unusual model.	High	Medium	Live-heap margin, bounded estimates, serialized waves.	Add telemetry-based estimate refinement and fail-fast lease cancellation.
Python Doc2Vec memory is outside Java heap accounting.	High	Medium	Memory mapping and one-thread BLAS defaults.	Add process RSS monitoring or node-count guidance for Doc2Vec.
K-Means/MLP remain intrinsically expensive on very wide corpora.	High	Medium	Sparse inputs and scheduling reduce peak overlap.	Add representative end-to-end million-row benchmarks per algorithm.
Dirty worktree contains binary/cache/LFS ambiguities.	High	High	Git status and artifact inventory captured.	Repair ignore rules and validate Gephi JAR through a clean LFS checkout.
Custom Swing GUI lacks automated visual/accessibility regression tests.	Medium	Medium	Manual screenshots and functional tests.	Add headless component-state tests and a release screenshot checklist.
Run lifecycle retains only sixteen retired IDs.	Medium	Low	UUID run IDs and current/retired admission.	Increase/age-limit history if exceptionally delayed JADE messages are observed.
Mapping cache uses path, mtime, and length.	Medium	Low	Atomic mapping writes and synchronized access.	Include a lightweight content hash if external edits are common.
Doc2Vec retraining is not bitwise deterministic.	Medium	Medium	Cache reuse preserves a fixed trained model.	Record cache signature/model artifact for experimental reproducibility.
Build script rejects Python 3.11+.	Medium	High	Direct compilation and shell launcher work.	Modernize `build.py` version checks and dependency support.

17.1 Recommendation Integrity Summary

Change class	Can it change ranked recommendations?	Reason
GUI layout, file picker, status bar	No	Presentation and input selection only.
Blocking JADE receives and lifecycle IDs	Not by design	Message timing/ownership changes; score formulas do not.
Sparse ARFF and sparse lookup	Not by design	Equivalent nonzero feature values; tests compare model behavior.
Adaptive concurrency	Not by design	Controls when model work runs, not its parameters.
Doc2Vec batch with fixed cache	No intended change per target	Calls the same shard scorer with each target independently.
Algorithm Default target mode	No intended change	Bypasses cross-type conversion.
Explicit Similar Users / Followee Accounts	Yes, intentionally	Changes the recommendation objective and may use hybrid conversion.
User generation rewrite	Yes for generated corpus	Creates a new synthetic dataset; it does not claim bitwise compatibility with legacy generation.
Post-processing rewrite	Ranked list no; persisted edge can vary	Ranked scores are already calculated; weighted random edge selection remains.

18. Release and Research Recommendations

18.1 Immediate Release Work

9. Freeze the current working tree and create a source-only inventory of intended Java, Python, shell, test, and documentation files.
10. Exclude generated corpora, Results, ARFF files, caches, compiled classes, `.DS_Store`, Python bytecode, and transient graph artifacts.
11. Resolve the Gephi toolkit JAR/LFS state before staging any binary.
12. Split the local work into reviewable commits: GUI/file picker; recommendation modes; graph/lifecycle/idle fixes; user generation; million-row representation; memory/heap; post-processing; tests.
13. Build and run tests from a clean clone of the resulting branch, not from the current artifact-rich workspace.
14. Push only after comparing `git diff --stat`, `git status`, and a generated release manifest.

18.2 Experimental Method Recommendations

- Record dataset checksum, algorithm, target mode, batch size, node count, heap, model cache signature, graph checkbox, and random seed for every experiment.
- Do not compare community counts across runs unless edge-selection randomness and Gephi modularity settings are fixed.
- For Doc2Vec single-versus-batch equivalence, reuse the same cached model and compare complete per-target score maps before graph generation.
- For SVM retail/Twitter reporting, state explicitly that field 0 is the predicted class and describe what field 0 means in that corpus.
- Treat 100 as a relative maximum after normalization, not automatically as confidence.
- Run a complete algorithm-by-algorithm benchmark matrix on representative small, original, and one-million-row corpora before claiming universal million-row support.

18.3 Recommended Performance Matrix

Dimension	Suggested levels
Corpus	Original Reduced_94k; Retail50K; generated 1M Twitter; generated 1M retail-like.

Dimension	Suggested levels
Algorithm	Similarity; K-Means; K-Means Euclidean; SVM; Doc2Vec; MLP.
Nodes	1, 2, 4, 8, and adaptive-limited result.
Heap	1.5 GB, 4 GB, 8 GB, and a host-appropriate upper setting.
Mode	Algorithm Default; Similar Users; Followee Accounts where supported.
Metrics	Wall time, peak Java heap, Python RSS, CPU, GC time, result completeness, score-map equality, edge count.

Appendix A. Change Ledger

ID	Work item	Result	State
A1	SVM batch and reproducibility baseline	Independent target training/scoring, deterministic seeds, batch target selection.	Committed/pushed
A2	Graph builder baseline	Validated graph/GML construction extracted and tested.	Committed/pushed
A3	Shared Doc2Vec infrastructure	Reusable model/scoring foundation introduced.	Committed/pushed
A4	GUI startup repair	Launcher corrected; PDFBox runtime bundled.	Committed/pushed
A5	Doc2Vec batch	Independent target maps, one Python load per node, structured protocol.	Committed/pushed + local hardening
A6	Reset Experiment	Dedicated reset button and transient artifact cleanup.	Committed/pushed + local expansion
A7	GUI modernization	Three-column layout, grouped controls, modern component system.	Workspace only
A8	File dialog	Custom searchable asynchronous picker; native dialog crash avoided.	Workspace only
A9	Initialize state	Enable after non-empty user load; consistent state transitions.	Workspace only
A10	Recommendation cards	Structured per-target ranked result display.	Workspace only
A11	Recommendation target modes	Default/users/accounts with typed conversion and validation.	Workspace only
A12	Graph checkbox	Skip Gephi while preserving accumulated edge state.	Workspace only
A13	Run lifecycle	UUID run IDs and stale-message rejection.	Workspace only
A14	Idle CPU	Blocking receives, templates, async popup, Gephi close.	Workspace only
A15	Data source/status redesign	Source cards and informative progress pipeline.	Workspace only
A16	User Generation Engine	Exact, atomic, streaming million-row generation.	Workspace only
A17	Streaming ingestion/index	Single-pass metadata and per-user tweet index.	Workspace only
A18	Sparse model inputs	Sparse ARFF, sparse point access, lower peak data duplication.	Workspace only
A19	Doc2Vec memory	Streaming corpus, normalized vector cache, NumPy memory mapping.	Workspace only
A20	MLP model averaging	One-model-at-a-time legacy-compatible aggregation.	Workspace only
A21	Adaptive model concurrency	Heap-aware safe waves with parallel execution when capacity permits.	Workspace only
A22	Heap UI and restart	Persisted max heap and safe application relaunch.	Workspace only
A23	Post-processing	Context-based edge persistence without repeated corpus scans/rewrite.	Workspace only
A24	Score semantics diagnosis	Explained top=100 normalization and relative scores.	Analysis only
A25	Retail/Twitter mismatch diagnosis	Explained user ranking versus field-0 account classification.	Analysis only
A26	Community-count diagnosis	Separated batch-score correctness from edge/community outcomes.	Analysis only
A27	Professor email draft	Requested clarification of intended recommendation entity.	Communication only

Appendix B. Principal Source Files

File	Role after modernization
`ControllerAgent.java`	Streaming dataset load, configuration validation, dataset context, agent startup, reset, memory status.
`ControllerAgentGui.java`	Modern workspace, source controls, result cards, runtime resources, graph and target-mode controls.
`RecommenderAgent.java`	Six algorithm paths, batch execution, typed payloads, sparse preparation, model leases.
`OrganizingAgent.java`	Per-run score merge, typed conversion merge, completion delivery.
`MobileAgent.java`	Blocking message flow, target query, structured result display, optimized post-processing.
`StarterAgent.java`	Run-scoped orchestration, optional graph stage, centralized completion.
`Doc2VecBatchRunner.java` / `doc2vec_batch.py`	Structured multi-target Doc2Vec batch bridge.
`doc2vec_shared.py`	Streaming model training, cache signature, vector index, shard scoring.
`ExperimentResetFiles.java`	Known transient-output cleanup policy.
`Recommendation*` classes	Target modes, candidate types, payloads, merge policies, run lifecycle, memory coordination, post-processing.
`UserGenerationEngine.java`	Validated exact-count synthetic corpus generation and analysis.
`ModernFilePicker.java`	Searchable asynchronous cross-platform file selection.
`SparseArffWriter.java`	Weka sparse dataset serialization.
`RuntimeResourceSettings.java` / relaunch classes	Heap persistence, validation, and restart.

Appendix C. Complexity Summary

Operation	Legacy cost	Current cost	Comment
Get tweets for one user	O(N) scan	O(1 + k) indexed lookup	Index build remains O(N).
Post-process one recommendation target	Approximately O((k+2)N) reads + O(N) rewrite	Approximately O(k) map lookups + append	Fallback may perform one O(N) scan.
ARFF row storage	O(V) values per user row	O(nnz) values per user row	Header remains O(V).
Doc2Vec batch model load	Potentially repeated per target	Once per node process for the batch	Scoring remains O(T × candidates/node).
Model concurrency	All requested models or fixed cap	Capacity-dependent safe waves	Parallel when reservations fit.
Generated output memory	Potentially output-size dependent	Buffered streaming	Source models still consume memory.

Appendix D. Terminology

Term	Meaning in this system
Target user	The user for whom a recommendation is requested.
Candidate user	A person/user identity that may be ranked as similar.

Term	Meaning in this system
Followee account	The reference account or class stored in field 0 of the six-field text row.
Algorithm Default	The untouched native output entity for the selected algorithm.
Batch users	Number of selected target users processed in one SVM or Doc2Vec run.
Recommender node	A JADE recommender agent responsible for a shard of input/candidates.
Organizer	The agent that merges distributed node scores.
Relative score	A merged value scaled so the maximum candidate is 100.
Graph output disabled	Skip Gephi visualization/export while retaining persisted nodes/edges.
Model lease	A memory reservation that admits an expensive model stage when capacity permits.

Appendix E. Final Assessment

The project moved from a research simulator that worked primarily on modest datasets and single-target workflows toward a more explicit experimental platform. The strongest improvements are correctness of Doc2Vec batching, clear experiment reset, typed recommendation semantics, safe preservation of graph history, removal of idle polling, exact atomic million-row generation, sparse representation, capacity-aware parallel model execution, and elimination of obsolete post-recommendation corpus rewrites.

The work does not make every algorithm universally cheap, deterministic, or semantically interchangeable. It does make limitations more visible and failure behavior safer. The next engineering milestone is not another feature: it is turning the current artifact-rich workspace into a clean, reviewable, reproducible release whose Git history matches the software actually being demonstrated.